Data Flow Testing

Course Code: CSC4133 Course Title: Software Quality and Testing

Dept. of Computer Science
Faculty of Science and Technology

Lecturer No:	**16**	**Week No:**	**9**	**Semester:**	
Lecturer:	*Prof. Dr. kamruddin Nur,* kamruddin@aiub.edu				

Lecture Outline

- The General Idea
- Data Flow Anomaly
- Data Flow Testing Criteria
- Feasible Path
- Comparison of Testing Techniques
- Summary

Objectives and Outcomes

- **Objectives**: To understand the basic concept of data flow testing, to understand three different types of data flow anomalies, to understand data flow testing criteria, to understand the effectiveness of different types of testing.

- **Outcomes**: Students are expected to be able to explain data flow testing, be able to explain three different types of data flow anomalies, be able to explain the data flow testing criteria, be able to compare the effectiveness of data flow testing with random testing and control flow testing.

The General Idea

- A program unit accepts inputs, performs computations, assigns new values to variables, and returns results.
- One can visualize of "**flow**" of **data values** from one statement to another.
- A data value produced in one statement is expected to be used later.
- Example:
  - Obtain a file pointer ……. use it later
  - If the later use is never verified, we do not know if the earlier assignment is acceptable

The General Idea

- Two motivations of data flow testing
  - The memory location for a variable is accessed in a "desirable" way.
  - Verify the correctness of data values "defined" (i.e. generated) – observe that all the "uses" of the value produce the desired results.
- Idea: A programmer can perform a number of tests on data values.
  - These tests are collectively known as 'data flow testing'.

The General Idea

- Data flow testing is a form of structural testing and a white box testing technique that focuses on program variables and the paths:
  - From the point where a variable, *v*, is defined or assigned a value
  - To the point where that variable, *v*, is used

The General Idea

- Data Flow Testing (DFT) looks at the life cycle of a data (or variable), checks if the usage may cause failure or anomaly, such as "definition is missing", "unreachable code", "incorrect assignment of input statement", etc.

The General Idea

- Data flow testing can be performed at two conceptual levels.
  - **Static** data flow testing
  - **Dynamic** data flow testing

- ***Note***: For this course, we will cover only **static data flow testing**

Static Data Flow Testing

- **Static data flow testing:**
  - Analyze source code.
  - Does not involve actual execution of source code.
  - Identify potential defects, commonly known as *data flow anomaly.*

Data Flow Anomaly

- Anomaly: An anomaly is a deviant or abnormal way of doing something.
- *Data-flow anomalies* represent the patterns of data usage which may lead to an incorrect execution of the code.
- Example 1: The second definition of *x* overrides the first.

  x = f1(y);

  x = f2(*z*);

Data Flow Anomaly

- *Examples* of data flow anomaly:
  - It is an abnormal situation to successively assign two values to a variable without using the first value.
  - It is abnormal to use a value of a variable before assigning a value to the variable.
  - Another abnormal situation is to generate a data value and never use it.

Data Flow Anomaly

- Three types of abnormal situations concerning the generation and use of data values:
  - Type 1: Defined and then defined again
  - Type 2: Undefined but referenced
  - Type 3: Defined but not referenced

Data Flow Anomaly

- Type 1: Defined and then defined again (*Example 1*)
  - Four interpretations of Example 1
    - The first statement is redundant.
    - The first statement has a fault -- the intended one might be: w = f1(y).
    - The second statement has a fault – the intended one might be: v = f2(z).
    - There is a missing statement in between the two: v = f3(x).
- *Note*: It is for the programmer to make the desired interpretation.

Data Flow Anomaly

- Type 2: Undefined but referenced :

  A second form of data flow anomaly is to *use an undefined variable in a computation,*

- **For example**: x = x – y – w; /* *w* has not been initialized by the programmer. */
  - Two interpretations:
    - The programmer made a mistake in using w.
    - The programmer wants to use the compiler assigned value of w.
  - One must eliminate the anomaly either by initializing w, or replacing w with the intended variable.

Data Flow Anomaly

- Type 3: Defined but not referenced

  A third kind of data flow anomaly is to define a variable and then to undefine it without using it in any subsequent computation.

  – For Example: Consider the statement $x = f(x, y)$ in which a new value is assigned to the variable x.

  – If the value of x is not used in any subsequent computation, then we have a Type 3 anomaly.

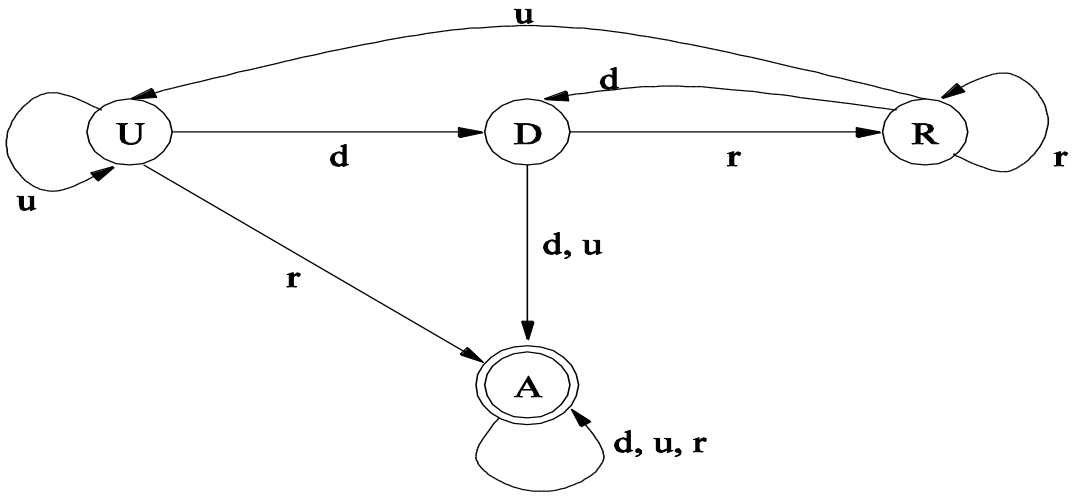
Data Flow Anomaly

- The concept of a state-transition diagram is used to model a **program variable** to identify data flow anomaly.
- Key idea: "states" of program variables can be used to identify data flow anomaly.
- Components of the state-transition diagrams:
  - **States**
  - **Actions**

Data Flow Anomaly

- The **states**
  - **U**: Undefined
  - **D**: Defined but not referenced
  - **R**: Defined and referenced
  - **A**: Abnormal
- The **actions**
  - ***d***: define the variable
  - ***r***: reference/read the variable
  - ***u***: undefine the variable

Data Flow Anomaly



Legends:

States
U: Undefined
D: Defined but not referenced
R: Defined and referenced
A: Abnormal

Actions
d: Define
r: Reference
u: Undefine

Figure 2: **State transition diagram** of a program **variable**

Data Flow Anomaly

- Initially, a variable can remain in an "undefined" (*U*) state, meaning that just a memory location has been allocated to the variable but no value has yet been assigned.
- At a later time, the programmer can perform a computation to define (d) the variable in the form of assigning a value to the variable –this is when the variable moves to a "defined but not referenced" (*D)* state.

Data Flow Anomaly

- At a later time, the programmer can reference (r), that is, read, the value of the variable, thereby moving the variable to a "defined and referenced" state *(R)* . The variable remains in the R state as long as the programmer keep referencing the value of the variable.
- If the programmer assigns a new value to the variable, the variable moves back to the *D* state. On the other hand, the programmer can take an action to undefine (u) the variable.

Data Flow Anomaly

- However, the programmer can make mistakes by taking the wrong actions while a variable is in a certain state. **For example**,
- If a variable is in the state *U* –that is the variable is still undefined –and a programmer reads (r) the variable, then the variable moves to an abnormal (*A*) state.
- Similarly, while a variable is in the state *D* and programmer undefines (u) the variable or redefines (d) the variable, then the variable moves to the abnormal (*A*) state.

Data Flow Anomaly

- Once a variable enters the abnormal state (*A*), it remains in that state irrespective of what action – *d, u,* or *r* is taken.

- The abnormal state of a variable means that a programming anomaly has occurred.

Books

- *Software Testing and Quality Assurance: Theory and Practice*, by Kshirasagar Naik, Priyadarshi Tripathy

References

1. *Software Quality Engineering: Testing, Quality Assurance and Quantifiable Improvement*, by Jeff Tian
2. *Software Quality Assurance: From Theory to Implementation*, by Daniel Galin
3. *Software Testing and Continuous Quality Improvement*, by William E. Lewis
4. *The Art of Software Testing*, by Glenford J. Myers, Corey Sandler and Tom Badgett